

TECNOLÓGICO DE MONTERREY

Maestría en Inteligencia Artificial Aplicada

TC5061 Operaciones de Inteligencia Artificial

Pruebas automatizadas y seguridad para MiBici

Optimización operativa de la red MiBici Guadalajara
2025 y 2026

Proyecto integrador
Actividad de la semana 7

Equipo 30

Ricardo Bahamón
A01840023

Diego Becerril
A01748115

Gregorio Escobedo Straffon
A01840612

Freddin Alcántara
A01797741

2 de noviembre de 2026

Repositorio del proyecto

https://github.com/Diegob059/TC5061-Proyecto-Equipo30

1 Pruebas y reproducibilidad

Se integraron al proyecto controles automatizados de código, calidad de datos, reproducibilidad y seguridad. Este reporte conserva los resultados de la ejecución del 2 de noviembre de 2026, junto con su evidencia y sus limitaciones.

54 pruebas aprobadas, 0 fallos, 0 errores y 0 omitidas. La suite cubre limpieza, variables temporales, modelos P1/P2/P3 y registro. Las 15 pruebas de validación de datos incluyen fronteras temporales, datos sin target, muestras inválidas, repetibilidad del muestreo y detección de duplicados y drift introducidos deliberadamente.

P2 se entrenó dos veces con semilla 42 sobre datos reales preparados como en producción, con 30 árboles por modelo y evaluación en Test. Coincidieron predicciones, probabilidades, F1 macro y balanced accuracy. P3 comprobó GMM con una fixture sintética identificada en JUnit; no se presenta esa corrida como validación con datos reales de P3. La prueba base de LightGBM también usa datos sintéticos.

```
make requirements
make test
```



Figura 1. Captura del visor de resultados obtenidos del XML y log reales de PyTest. Fuentes: reports/testing/pytest.xml y pytest.log.

2 DeepChecks y separación temporal

Se reconstruyó el dataset de P2 desde los **21 CSV mensuales reales** de enero de 2025 a septiembre de 2026: **7,196,640 filas**. Los datos se conservan localmente y están excluidos de Git. El manifiesto reports/testing/data_manifest.json registra los hashes de origen.

La auditoría reutiliza **get_train_test_data** del entrenamiento: excluye el calentamiento, target=-1 y las variables futuras ratio/delta_net. Compara Train con timestamp < 2025-12-31 22:00:00 y Test desde 2026-01-01. Se muestrean después 7,500 filas por partición, con semilla 42, y se evalúan **17 variables predictoras**. Fecha e identidad estación/hora se mantienen como metadatos.

Suite	Ejecutados	Incumplidas	Sin ejecutar
Integridad de datos	12	0	0
Validación Train/Test	12	0	0

Ambas suites aprobaron sus condiciones. Outlier Sample Detection es descriptivo en la suite estándar y no tiene umbral de aprobación. Los hallazgos antiguos sobre ratio, delta_net y la etiqueta -1 provenían de una preparación distinta de la usada para entrenar; los informes actuales se generaron juntos a partir del mismo Parquet.

```
make data
make deepchecks
python -m scripts.run_deepchecks --sample-size 15000
```

Los HTML se exportan sin depender de widgets externos. El CLI devuelve 0 al aprobar, 1 si hay condiciones incumplidas o chequeos sin ejecutar y 2 para datos o argumentos inválidos. Guarda HTML y JSON también cuando hay hallazgos; generar un informe no se interpreta como aprobación.



Figura 2. Captura del visor de las suites reales. Los resultados corresponden a la muestra auditada; no garantizan la calidad de todas las filas ni de datos futuros.

Fuentes: reports/deepchecks_summary.json, deepchecks_data_integrity.html, deepchecks_data_drift.html y reports/testing/deepchecks.log. El JSON incluye columnas, versiones y SHA-256 del Parquet.

3 Seguridad: resultados y pendientes

Bandit: 5,159 líneas analizadas, 0 hallazgos altos, 0 medios y 23 bajos (B404, B603 y B607). Se revisaron los subprocesos de Git con argumentos fijos y sin shell=True. Los avisos se mantienen visibles en el informe; el comando devuelve 1 al detectarlos.

pip-audit: 81 avisos conocidos en nueve paquetes del entorno revisado, incluidas herramientas como pip. El recuento corresponde a las versiones instaladas y a la base consultada en la fecha de ejecución. No se afirma que estos avisos estén mitigados.

Paquete	Versión	Avisos	Tratamiento pendiente
cryptography	46.0.7	7	Migrar MLflow: actualmente exige <47.
diskcache	5.6.3	2	El escáner no informa una versión corregida.
fastmcp	2.14.7	4	Migrar MLflow: actualmente exige <3.
joserfc	1.7.5	1	Evaluar 1.7.6 o posterior y probar.
mlflow	3.5.0	47	Actualizar y validar tracking, modelos y Registry.
pip	25.0.1	12	Actualizar la herramienta y repetir auditoría.
pyarrow	21.0.0	2	Migrar MLflow: actualmente exige <22.
setuptools	80.10.2	3	Resolver pkg_resources de DeepChecks primero.
tornado	6.5.10	3	Evaluar 6.6 o posterior y probar.

Se conserva pyasn1==0.6.5, corrección aplicada por el equipo en el PR #29; no presentó hallazgos en esta auditoría. Se fija setuptools==80.10.2 para que DeepChecks 0.19.1 pueda importar pkg_resources. Esta compatibilidad resuelve el arranque, pero conserva deuda de seguridad. La migración de MLflow y sus dependencias continúa pendiente.

Secretos y datos: .gitignore excluye .env, datos, modelos y bases de MLflow; la configuración utiliza variables de entorno. La búsqueda de patrones en el código actual y de nombres de archivos sensibles en ramas locales no encontró coincidencias. No se analizó todo el contenido histórico ni se garantiza ausencia de secretos.

```
make security_code
make security_deps
```

Fuentes: reports/testing/bandit.json, pip-audit.json, secrets-review.json, environment.txt y execution.json. El JSON de pip-audit conserva los identificadores y las versiones corregidas propuestas por el escáner.

4 Evidencia y trabajo colaborativo

El historial de Git registra las contribuciones del equipo mediante los siguientes pull requests y commits, vinculados con los archivos modificados y la evidencia de ejecución.

Integrante	Aportación identificable
Freddin Alcántara	PR #26: prueba de reproducibilidad de LightGBM.
Ricardo Bahamón	PR #27, #28 y #30: reproducibilidad, DeepChecks y documentación.
Diego Becerril	PR #29: chequeos de seguridad y ajuste de pyasn1.
Gregorio Escobedo Straffon	Commits 4e7a232 y a148bf0: validación alineada con P2, regresiones, compatibilidad y exportación HTML independiente. Rama corregir-deepchecks-semana7.

Los archivos reports/testing/execution.json y environment.txt conservan comandos, códigos de salida, fechas con zona horaria, versiones y hashes del código probado. PyTest y DeepChecks devolvieron 0; Bandit y pip-audit devolvieron 1 por los hallazgos descritos. Los logs completos y los JSON/XML permanecen disponibles en el repositorio.

Las figuras corresponden a **capturas de los visores de los archivos de ejecución**. Sus fuentes están identificadas en los pies de figura y disponibles en el repositorio para contrastar los resultados.



Figura 3. Captura del visor de seguridad. Los avisos de dependencias permanecen abiertos y se distinguen de los resultados del análisis estático.

Las pruebas automatizadas y las condiciones de las suites de datos se cumplieron en la ejecución registrada. El análisis de seguridad conserva los hallazgos descritos en la sección 3 y los archivos que permiten verificarlos.